

The background features a light gray hexagonal grid pattern. Overlaid on this is a blue-tinted image of a microchip and a mechanical component, possibly a lens or a part of a microscope. Small green and blue triangles are scattered across the grid.

SoC101 | 2026-1

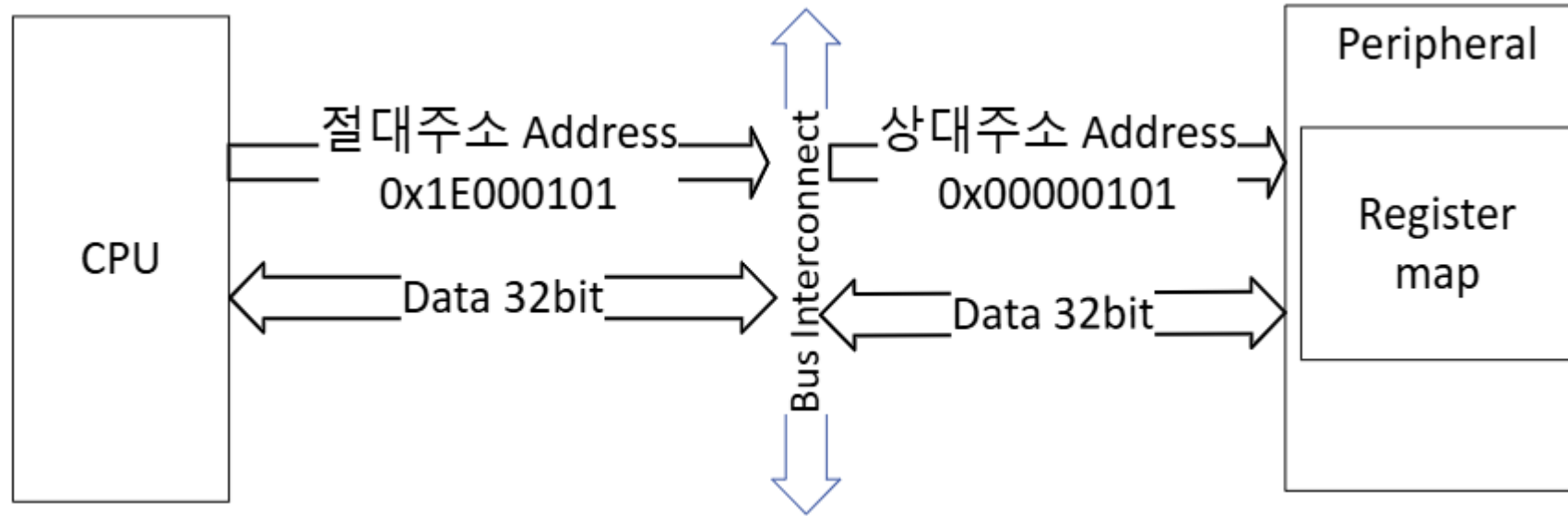
Session7 : Sequential Logic Blocks(2)

Agenda

- Basic Sequential Blocks
 - Register Map / Memory
- Understanding Finite State Machine(FSM)
 - Moore Machine
 - Mealy Machine
- LABS

1. Register Map & Memory

Register Map



레지스터 맵(Register Map) 예시

주소 (Hex)	이름	설명	읽기/쓰기
0x00	CONTROL	하드웨어 제어	RW
0x04	STATUS	하드웨어 상태	R
0x08	DATA_IN	입력 데이터	R
0x0C	DATA_OUT	출력 데이터	W

Memory Map

- 메모리 맵(Memory Map)
- ****메모리 맵****은 SoC에서 사용 가능한 메모리 공간을 정의한 것입니다.
 - CPU가 접근할 수 있는 모든 메모리 영역을 포함하며, 각 영역은 특정 용도로 나뉩니다.
- 특징
 - 주소 공간 분할: 메모리는 코드, 데이터, 스택, 외부 장치 등으로 분할됩니다.
- 장치 매핑:
 - 외부 장치(예: UART, SPI 등)는 메모리 주소에 매핑되어 CPU가 이를 제어할 수 있습니다.
- 효율적 설계:
 - 메모리 맵을 통해 데이터와 장치 간의 효율적인 접근이 가능합니다.

예시

주소 범위 (Hex)	용도	설명
0x00000000~0x0000FFFF	ROM	부트 코드 저장
0x00010000~0x0001FFFF	RAM	실행 중인 프로그램 데이터
0x80000000~0x80000FFF	UART	UART 제어 및 데이터 전송
0x90000000~0x90000FFF	GPIO	GPIO 핀 설정 및 상태 확인

Memory Management with Register Map

- 레지스터 맵(Register Map)은 하드웨어 모듈의 제어 및 상태 정보를 특정 메모리 주소에 매핑하여 소프트웨어가 이를 효율적으로 관리
- 하드웨어와 소프트웨어 간의 상호작용이 체계적으로 이루어지며, 메모리 관리가 간소화
- 레지스터 맵을 통한 메모리 관리의 주요 원칙

원칙	설명
주소 공간 분할	각 레지스터에 고유한 메모리 주소를 할당하여 하드웨어 모듈 간 충돌을 방지하고 체계적으로 관리.
효율적인 접근	CPU가 특정 주소를 통해 하드웨어를 제어하거나 상태를 읽음으로써 빠르고 간단한 데이터 처리 가능.
읽기/쓰기 권한 설정	레지스터마다 읽기(Read), 쓰기(Write), 읽기-쓰기(Read-Write) 권한을 설정하여 잘못된 접근 방지.
확장성	새로운 하드웨어 모듈 추가 시 기존 메모리 구조에 쉽게 통합 가능 (새로운 주소 공간만 추가).
메모리 보호	권한 설정 및 주소 분리를 통해 소프트웨어가 불필요하거나 잘못된 영역에 접근하지 않도록 보호.

Memory Management with Register Map

항목	설명
효율적인 자원 분배	각 하드웨어 모듈에 고유한 메모리 공간을 할당하여 충돌을 방지하고, CPU가 효율적으로 접근 가능.
소프트웨어와 하드웨어 통합	소프트웨어가 특정 메모리 주소를 통해 하드웨어를 제어하거나 상태를 읽음으로써 간단한 통합 가능.
확장성	새로운 하드웨어 모듈 추가 시 기존 구조에 쉽게 통합 가능 (새로운 주소 공간만 추가).
읽기/쓰기 권한 설정	레지스터마다 읽기(Read), 쓰기(Write), 읽기-쓰기(Read-Write) 권한을 설정하여 메모리 보호 가능.
디버깅 및 유지보수 용이성	명확히 정의된 레지스터 맵 덕분에 소프트웨어와 하드웨어 간의 인터페이스가 직관적이고 문제 해결이 용이.

LAB: DUT

LAB : DUT

```
1  `timescale 1ns / 1ps
2  module SimpleRegister (
3      input  clk,           // 클럭 신호
4      input  reset,         // 리셋 신호
5      input  write_enable,   // 쓰기 활성화 신호
6      input  [7:0] write_data, // 쓰기 데이터
7      output [7:0] read_data // 읽기 데이터
8  );
9
10     // 8비트 레지스터 선언
11     reg [7:0] register;
12
13     // 클럭 상승 에지에서 동작
14     always @(posedge clk or posedge reset) begin
15         if (reset) begin
16             register <= 8'b0; // 리셋 시 레지스터 초기화
17         end else if (write_enable) begin
18             register <= write_data; // 쓰기 활성화 시 데이터 저장
19         end
20     end
21
22     // 읽기 데이터는 항상 현재 레지스터 값 출력
23     assign read_data = register;
24
25 endmodule
```

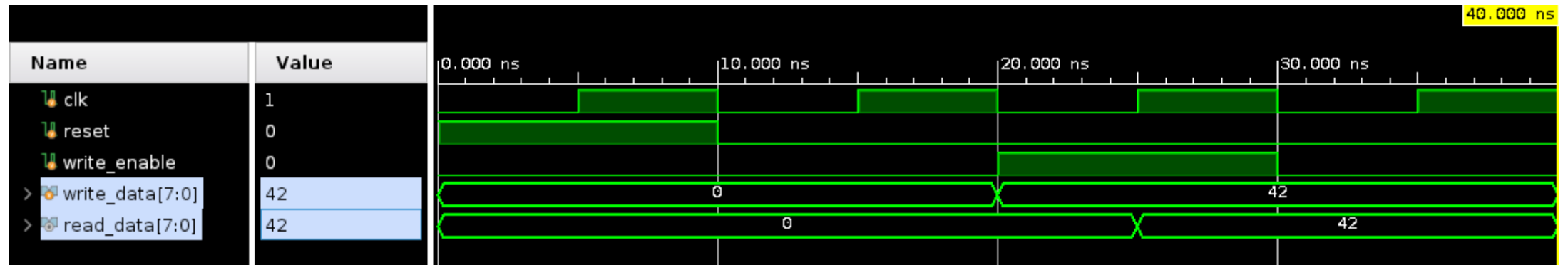
LAB : Testbench



```
1  `timescale 1ns / 1ps
2  module Testbench;
3
4      reg clk;
5      reg reset;
6      reg write_enable;
7      reg [7:0] write_data;
8      wire [7:0] read_data;
9
10     // SimpleRegister 인스턴스 생성
11     SimpleRegister uut (
12         .clk(clk),
13         .reset(reset),
14         .write_enable(write_enable),
15         .write_data(write_data),
16         .read_data(read_data)
17     );
```

```
19     // 클럭 생성기
20     initial begin
21         clk = 0;
22         forever #5 clk = ~clk; // 10ns 주기의 클럭 생성
23     end
24
25     initial begin
26         // 초기화 및 테스트 시퀀스 시작
27         reset = 1; write_enable = 0; write_data = 8'b0;
28         #10 reset = 0; // 리셋 비활성화
29
30         // 쓰기 테스트: 값 42 저장
31         #10 write_enable = 1; write_data = 8'd42;
32         #10 write_enable = 0;
33
34         // 읽기 테스트: 저장된 값 확인
35         #10 $display("Read Data: %d", read_data);
36
37         $finish; // 시뮬레이션 종료
38     end
39
40 endmodule
```

LAB : Waveform

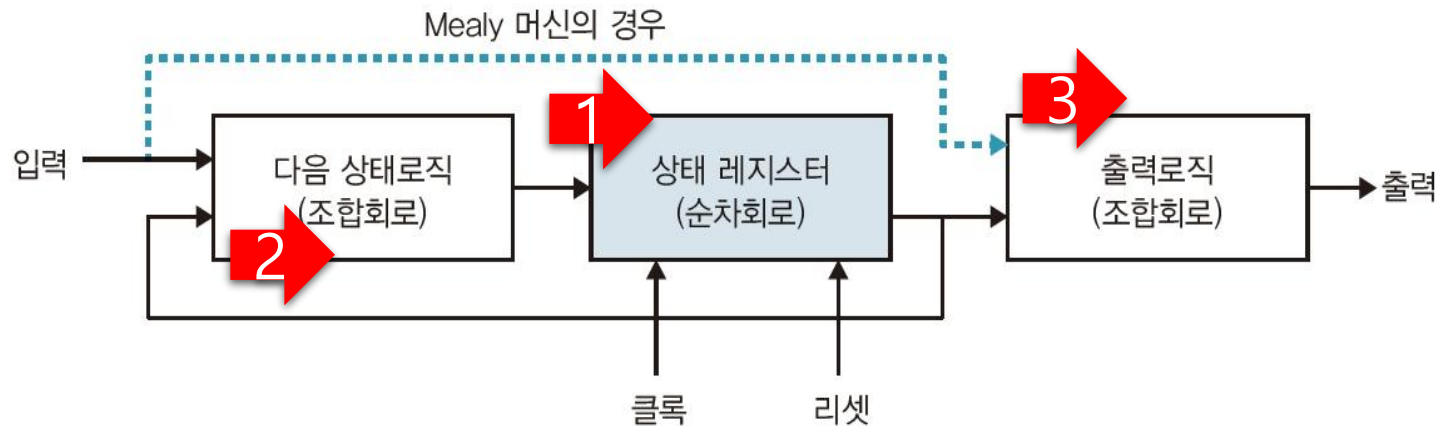


2. Finite State Machine

SoC 기본 유한상태머신 회로

- 유한상태 머신(Finite State Machine; FSM)

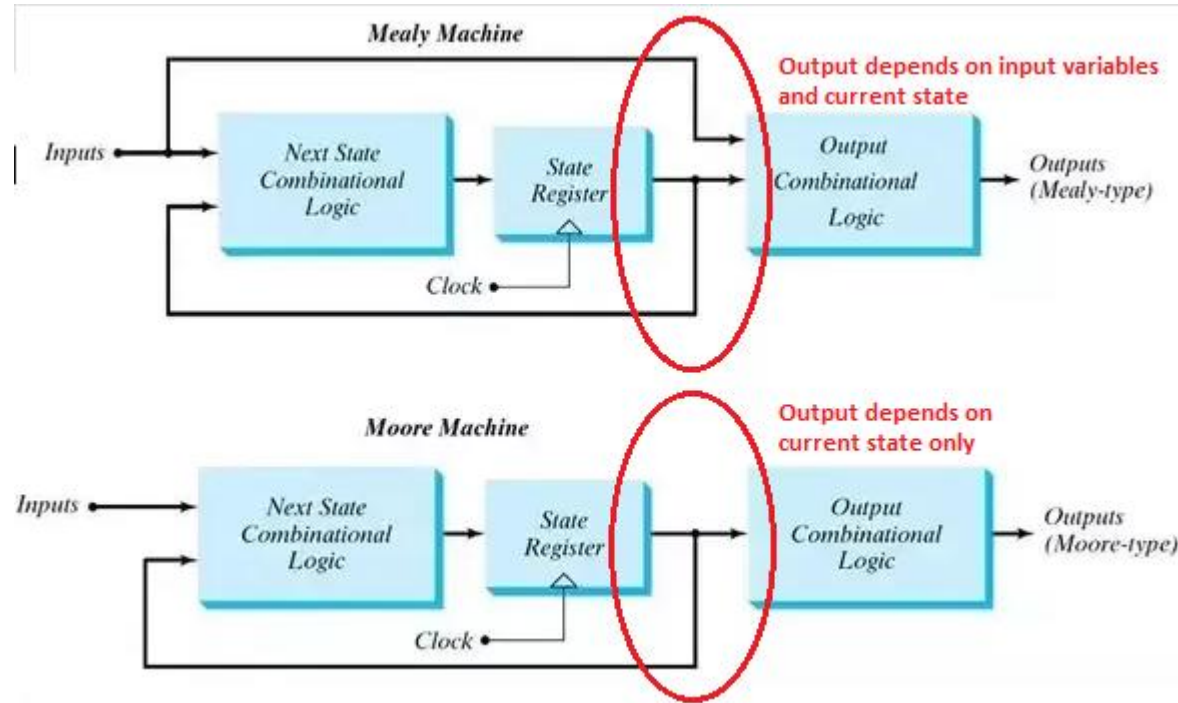
- 정해진 수의 상태를 가지고 상태들 간의 천이에 의해 출력을 생성하는 회로
- 디지털 시스템의 제어회로 구성에 사용
- Moore 머신** : 출력이 단지 현재상태에 의해서 결정
- Mealy 머신** : 현재상태와 입력에 의해 출력이 결정



[그림 11-27] 유한상태머신의 구조

로직 종류	역할 및 특징
1. State Sequential Logic	클럭을 기준으로 현재 상태(state)를 저장하고 업데이트합니다.
2. Next Combinational Logic	입력과 현재 상태에 따라 다음 상태(next_state)를 결정합니다.
3. Output Logic	Moore FSM에서는 현재 상태만으로 출력을 결정하며, Mealy FSM에서는 입력과 현재 상태 모두를 고려하여 출력을 결정합니다.

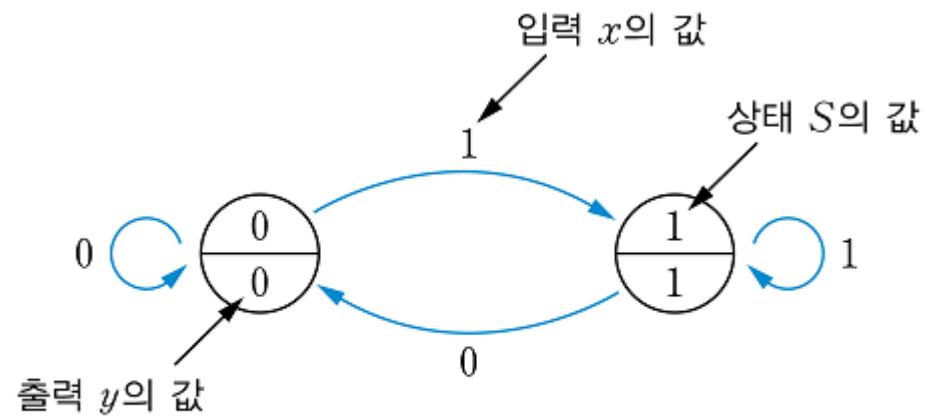
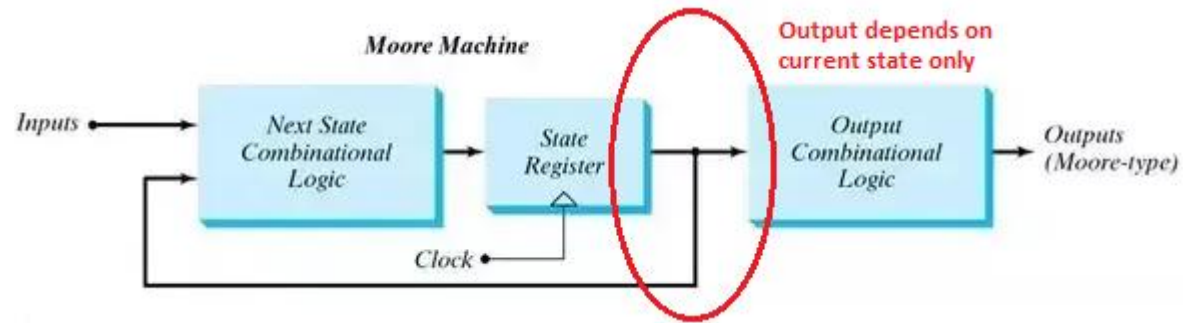
SoC 기본 유한상태머신 회로



특징	Mealy 머신	Moore 머신
출력 결정 방식	현재 상태와 입력	현재 상태
출력 반응 속도	입력 변화에 즉각 반응	클럭 주기에 따라 반응
설계 복잡도	상대적으로 복잡	상대적으로 단순
상태 수	적음	많음

2.1. Moore Machine

Moore Machine



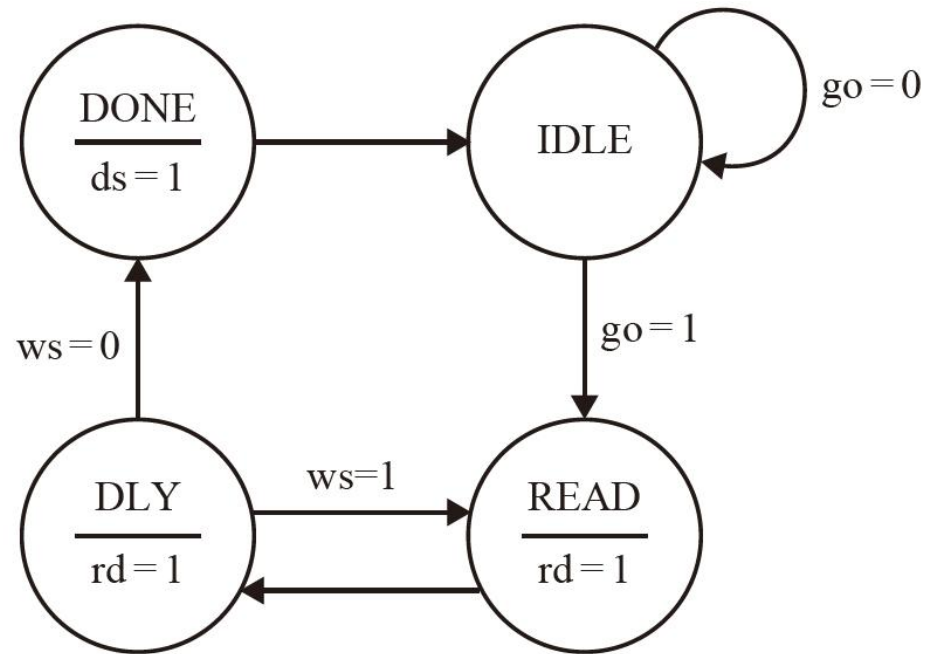
(d) 상태도

Encoding : state value

상태번호	이진 인코딩	Gray 인코딩	Johnson 인코딩	one-hot 인코딩
0	000	000	0000	00000001
1	001	001	0001	00000010
2	010	011	0011	00000100
3	011	010	0111	00001000
4	100	110	1111	00010000
5	101	111	1110	00100000
6	110	101	1100	01000000
7	111	100	1000	10000000

LAB : Specification & SM Chart

- 4개의 상태를 갖는 FSM 모델링



[그림 11-28] 4개의 상태를 갖는 FSM 예

현재 상태	입력 조건	다음 상태	출력
IDLE	go = 0	IDLE	없음
IDLE	go = 1	READ	없음
READ	ws = 1	DLY	rd = 1
DLY	ws = 0	DONE	rd = 1
DONE	N/A	IDLE	ds = 1

LAB : DUT

```
1  `timescale 1ns / 1ps
2  module moore_fsm (
3      input clk,           // 클럭 신호
4      input reset,         // 리셋 신호 (비동기)
5      input go,            // 입력 신호 go
6      input ws,            // 입력 신호 ws
7      output reg rd,       // 출력 신호 rd
8      output reg ds        // 출력 신호 ds
9  );
10
11  // 상태 정의 (4개의 상태)
12  localparam IDLE = 2'b00,
13             READ = 2'b01,
14             DLY  = 2'b10,
15             DONE = 2'b11;
16
17  reg [1:0] state, next_state; // 현재 상태와 다음 상태
18
19  // 상태 레지스터: 현재 상태를 저장
20  always @(posedge clk or posedge reset) begin
21      if (reset)
22          state <= IDLE;           // 리셋 시 초기 상태는 IDLE
23      else
24          state <= next_state; // 다음 상태로 전환
25  end
```

```
27  // 다음 상태 로직: 입력(go, ws)에 따라 상태 전환 결정
28  always @(*) begin
29      case (state)
30          IDLE:
31              if (go)
32                  next_state = READ; // go=1이면 READ로 전환
33              else
34                  next_state = IDLE; // go=0이면 IDLE 유지
35
36          READ:
37              if (ws)
38                  next_state = DLY; // ws=1이면 DLY로 전환
39              else
40                  next_state = READ; // ws=0이면 READ 유지
41
42          DLY:
43              if (!ws)
44                  next_state = DONE; // ws=0이면 DONE으로 전환
45              else
46                  next_state = DLY; // ws=1이면 DLY 유지
47
48          DONE:
49              next_state = IDLE; // DONE 이후에는 항상 IDLE로 복귀
50
51          default:
52              next_state = IDLE; // 기본값은 IDLE
53      endcase
54  end
```

LAB : DUT

```
57 // 출력 로직: 현재 상태에 따라 출력 결정 (Moore 머신)
58 always @(*) begin
59     case (state)
60         IDLE: begin
61             rd = 0;
62             ds = 0;
63         end
64
65         READ: begin
66             rd = 1; // READ 상태에서 rd 활성화
67             ds = 0;
68         end
69
70         DLY: begin
71             rd = 1; // DLY 상태에서 rd 유지 활성화
72             ds = 0;
73         end
74
75         DONE: begin
76             rd = 0;
77             ds = 1; // DONE 상태에서 ds 활성화
78         end
79
80         default: begin
81             rd = 0;
82             ds = 0;
83         end
84     endcase
85 end
86
87 endmodule
```

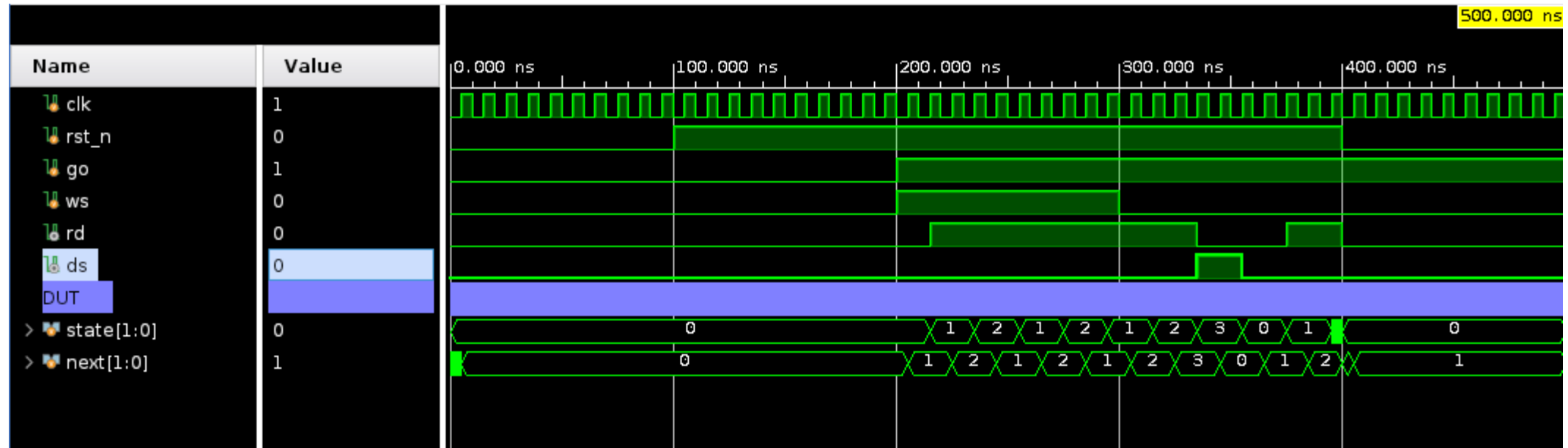


LAB : Testbench

```
1  `timescale 1ns / 1ps
2
3  module tb_moore_fsm;
4
5      // 테스트 벤치에서 사용할 신호 정의
6      reg clk;          // 클럭 신호
7      reg reset;        // 리셋 신호
8      reg go;           // 입력 신호 go
9      reg ws;           // 입력 신호 ws
10     wire rd;          // 출력 신호 rd
11     wire ds;          // 출력 신호 ds
12
13     // DUT (Device Under Test) 인스턴스화
14     moore_fsm uut (
15         .clk(clk),
16         .reset(reset),
17         .go(go),
18         .ws(ws),
19         .rd(rd),
20         .ds(ds)
21     );
22
23     // 클럭 생성: 10ns 주기로 클럭 신호 변경
24     always #5 clk = ~clk;
```

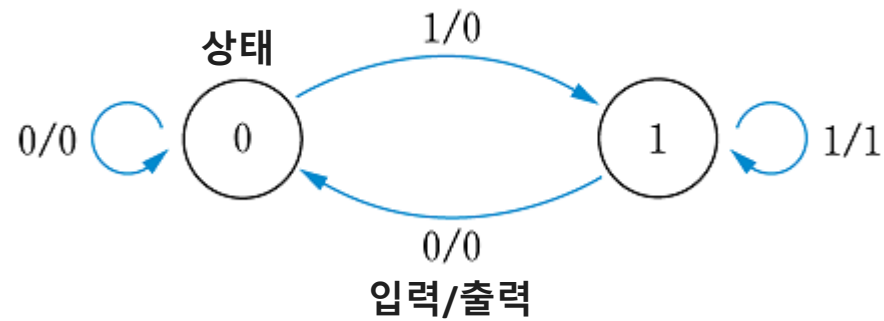
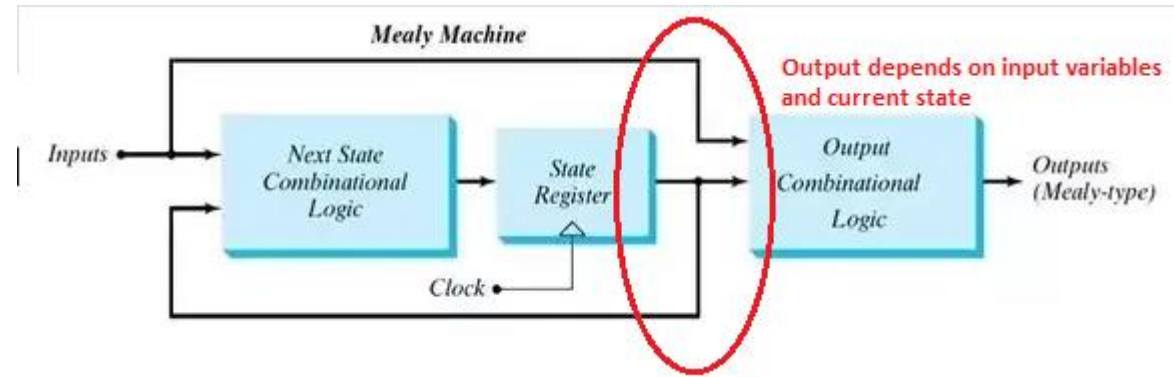
```
26     // 초기화 및 테스트 시퀀스 작성
27     initial begin
28         // 초기 상태 설정
29         clk = 0;
30         reset = 0;
31         go = 0;
32         ws = 0;
33
34         // 리셋 활성화 (비동기 리셋)
35         #10 reset = 1;
36         #10 reset = 0;
37
38         // IDLE -> READ 상태로 전환 (go=1)
39         #10 go = 1;
40         #10 go = 0;
41
42         // READ -> DLY 상태로 전환 (ws=1)
43         #10 ws = 1;
44
45         // DLY -> DONE 상태로 전환 (ws=0)
46         #20 ws = 0;
47
48         // DONE -> IDLE 상태로 복귀 (자동 복귀)
49         #20;
50
51         // 추가 테스트: 다시 READ 상태로 전환
52         #10 go = 1;
53         #10 go = 0;
54
55         // 종료 조건
56         #50 $stop;
57     end
58
59     // 모니터링: 출력 값 확인
60     initial begin
61         $monitor("Time=%0t | clk=%b | reset=%b | go=%b | ws=%b | rd=%b | ds=%b | state=%b",
62             $time, clk, reset, go, ws, rd, ds, uut.state);
63     end
64
65 endmodule
```

LAB : Waveform



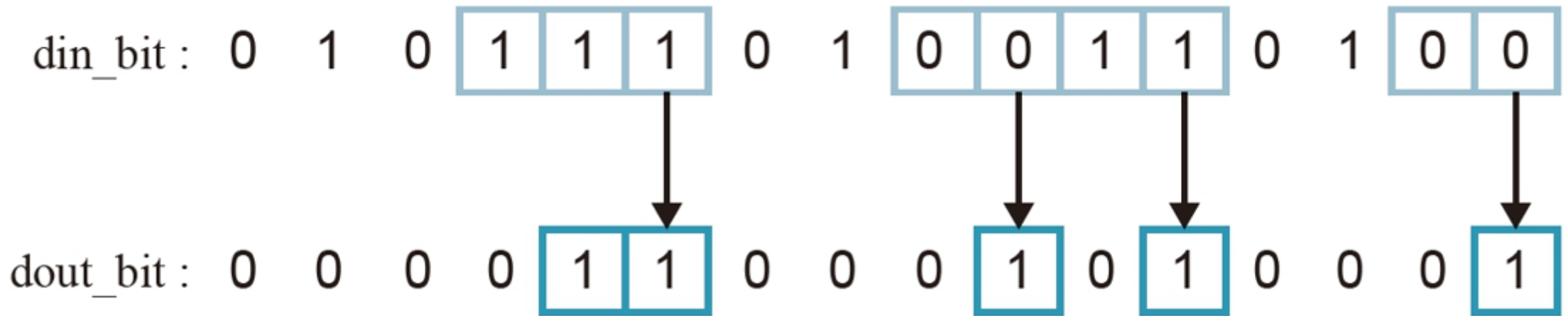
2.2. Mealy Machine

Mealy Machine

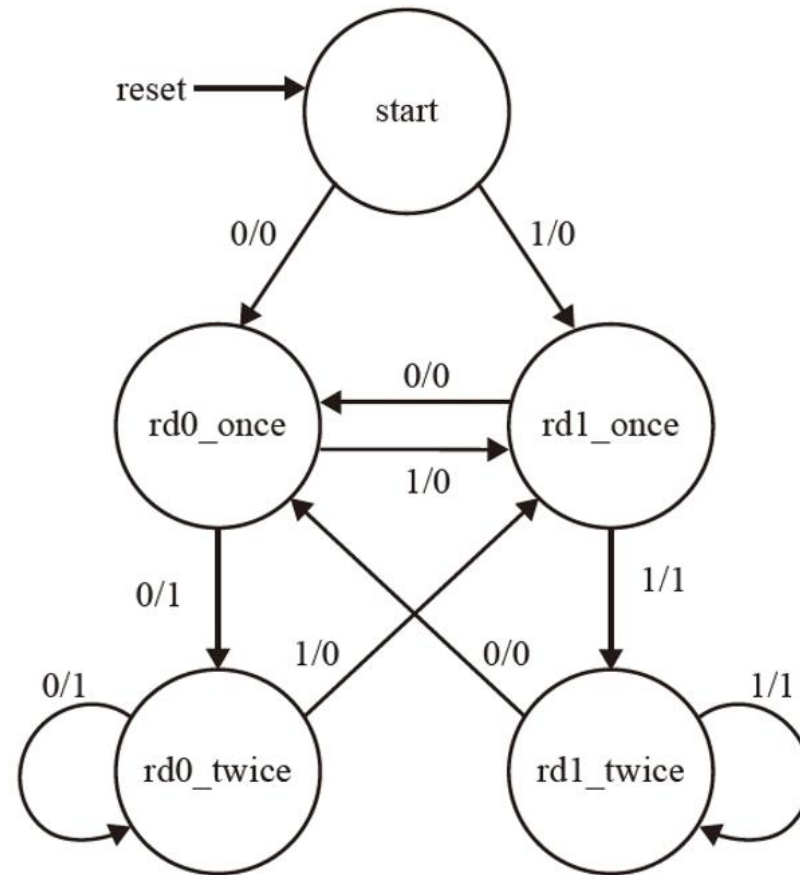


LAB: Specification

- Sequence Detector : detect consecutive 0 or 1 detector



LAB: SM Chart



LAB : DUT

1

```
1  `timescale 1ns / 1ps
2
3  module FSM (
4  // port list
5      clk      ,
6      rst_n    ,
7      din_bit  ,
8      dout_bit
9  );
10
11 // port declaration
12 input  clk      ;
13 input  rst_n    ;
14 input  din_bit  ;
15 output dout_bit ;
16
17 // parameters for each state
18 parameter START    = 3'b000 ;
19 parameter RD0_ONCE = 3'b001 ;
20 parameter RD1_ONCE = 3'b010 ;
21 parameter RD0_TWICE = 3'b011 ;
22 parameter RD1_TWICE = 3'b100 ;
23
24 // internal registers to store current state
25 reg [2:0] state ;
26 reg [2:0] next ;
27
28 // 1. current state register
29 always @(posedge clk, negedge rst_n) begin
30     if ( !rst_n ) state <= START;
31     else      state <= next;
32 end
```

3

4

```
28 // 1. current state register
29 always @(posedge clk, negedge rst_n) begin
30     if ( !rst_n ) state <= START;
31     else      state <= next;
32 end
33
34 // 2. next state logic
35 always @(state, din_bit) begin
36     case ( state )
37         START : if ( din_bit == 0 )      next = RD0_ONCE ;
38                 else if ( din_bit == 1 ) next = RD1_ONCE ;
39                 else                     next = START   ;
40         RD0_ONCE : if ( din_bit == 0 )    next = RD0_TWICE ;
41                   else if ( din_bit == 1 ) next = RD1_ONCE ;
42                   else                     next = START   ;
43         RD0_TWICE : if ( din_bit == 0 )    next = RD0_TWICE ;
44                    else if ( din_bit == 1 ) next = RD1_ONCE ;
45                    else                     next = START   ;
46         RD1_ONCE : if ( din_bit == 0 )    next = RD0_ONCE ;
47                   else if ( din_bit == 1 ) next = RD1_TWICE ;
48                   else                     next = START   ;
49         RD1_TWICE : if ( din_bit == 0 )    next = RD0_ONCE ;
50                   else if ( din_bit == 1 ) next = RD1_TWICE ;
51                   else                     next = START   ;
52         default : next = START ;
53     endcase
54 end
55
56 // 3. output logic(data flow modeling)
57 assign dout_bit = (((state == RD0_TWICE) && (din_bit == 0)) ||
58                  ((state == RD1_TWICE) && (din_bit == 1))) ? 1 : 0;
59
60
61 endmodule
```

LAB : Testbench

```
1  `timescale 1ns / 1ps
2
3  module tb_FSM ();
4
5  // signal def
6  // stimulus
7  reg clk;
8  reg rst_n;
9  reg din_bit;
10 // monitor signal
11 wire dout_bit;
12
13 // DUT instantiation
14 FSM DUT (
15     .clk      ( clk      ),
16     .rst_n    ( rst_n    ),
17     .din_bit  ( din_bit  ),
18     .dout_bit ( dout_bit )
19 );
20
21 // clock gen
22 always #5 clk = ~clk;
```

```
25 // apply stimulus
26 initial begin
27     // t=0, init
28     clk = 1'b0; rst_n = 1'b0; din_bit = 1'b0;
29
30     // reset release(system start)
31     #100 rst_n = 1'b1;
32     // put stimlus
33     #10 din_bit = 1'b0;
34     #10 din_bit = 1'b0;
35     #10 din_bit = 1'b1;
36     #10 din_bit = 1'b1;
37     #10 din_bit = 1'b1;
38     #10 din_bit = 1'b0;
39     #10 din_bit = 1'b0;
40     #10 din_bit = 1'b0;
41     #10 din_bit = 1'b0;
42     #10 din_bit = 1'b1;
43     #10 din_bit = 1'b1;
44     #10 din_bit = 1'b1;
45     #10 din_bit = 1'b1;
46     #10 din_bit = 1'b1;
47     #10 din_bit = 1'b0;
48     #100 rst_n = 1'b0;
49     #2000; $finish;
50 end
51
52 endmodule
```

LAB : Waveform

